

详解 21 · DMA 环形缓冲 —— 和普通模式的区别（以 ESP8266 接收为例）

覆盖代码： BSP/ESP8266/esp8266.c （接收那一层）、 BSP/ESP8266/esp8266.h （缓冲大小宏）、 Core/Src/usart.c （DMA 句柄配置）、 Core/Src/dma.c （NVIC）、 Core/Src/stm32f1xx_it.c （中断服务函数）

读者： 知道"DMA 能在不占 CPU 的情况下搬数据"，但没分清 **普通模式（NORMAL）** 和 **环形模式（CIRCULAR）**，以及为什么收串口一定要用后者的人。

阅读前提： 知道 UART / 寄存器 / 中断是什么。DMA 的基础操作（谁搬、搬到哪）本文从零讲。

本文不讲什么： AT 指令和 ESP8266 驱动接口（见 [详解 20](#)）；任务和队列（见 [详解 19](#)）。本项目的**发送**路径用的是中断不是 DMA，本文只在对比处提一句。

行号说明： 本文只写函数名和寄存器名，不写行号。

21.1 这块代码要解决什么问题

一句话

让串口收到的每一个字节都不会丢 —— 包括 CPU 正在关中断读 DHT11 的那 4 毫秒。

解法是把串口接收挂到 DMA 的**环形模式**上，让硬件自己往一个圈里写，CPU 有空的时候再去追。

问题：4 毫秒没看串口，字节就没了

三串数字决定了这件事必须用 DMA：

事实	数字
波特率	115200，8-N-1 → 一个字节 10 位
来一个字节要多久	$10 \div 115200 \approx$ 87 μs
读一次 DHT11 要关中断多久	约 4 ms

4 ms ÷ 87 μs ≈ **46 个字节**。如果用 RXNE 中断收，这 4 ms 里来的 46 个字节会一个不剩地丢掉 —— 而 AT 指令的应答、OneNET 的回应，恰好可能就是在这段时间里到的。**这种丢包不是"偶尔丢一个"，而是"每次读 DHT11 都丢一段"，症状是应答被判超时、随机失败。**

DMA 不管 CPU 在干什么：CPU 关中断、跑别的任务、睡死过去，DMA 都照搬不误。这就是唯一的出路。

约束

约束	后果
STM32F1 的串口 只有 1 个字节的接收寄存器，没有 FIFO	不搬走就会溢出（ORE），所以必须有人（DMA 或 CPU）及时取走
F1 的 DMA 通道是 硬连线 的	USART2_RX 只能是 DMA1_Channel6，不像 F4 有 DMAMUX 可以随便挑
DMA 是"外设每来一个字节请求一次"	不是"一次搬 11 个字节"，缓冲区里的字节是一个一个攒起来的
环形缓冲的存储是 RAM	256 字节的 s_DmaBuf + 512 字节的 s_RxBuf 都算在 ZI 段里
应答是 长度未知、源源不断 的流	这正是环形模式存在的理由（长度已知的一块才用普通模式）

本文不讲什么

- 驱动怎么发命令、怎么判应答 —— [详解 20](#)。
- 任务的调度、队列 —— [详解 19](#)。
- 发送路径为什么用中断不用 DMA：那是"长度已知、一次 256 字节"的活，中断够用，理由在 21.4。

21.2 整体结构

先分清三件事：DMA、通道、模式

DMA 是一块独立于 CPU 的搬运硬件：你告诉它"从哪搬、搬到哪、搬多少个"，它就在外设和内存之间自己搬，每搬完一个可以给 CPU 一个中断或不给。

一次 DMA 传输永远是三个量：

量	在本项目里	说明
源地址	USART2->DR （外设数据寄存器）	串口收到的每个字节都出现在这里。地址 不变 （所以配的是 DMA_PINC_DISABLE ）
目的地址	s_DmaBuf[]	每搬一个字节，地址 +1 （所以配的是 DMA_MINC_ENABLE ）
传输个数	CNDTR 寄存器	初值 = 要搬的个数， 每搬一个减 1

通道是 DMA 控制器里的一个"搬运工"。STM32F1 的 DMA1 有 7 个通道，每个通道固定绑死几个外设请求（USART2_RX 只能在通道 6）—— 这是硬件连线，不是软件可以挑的。

模式决定 CNDTR 减到 0 之后发生什么 —— **这就是本文的主题：**

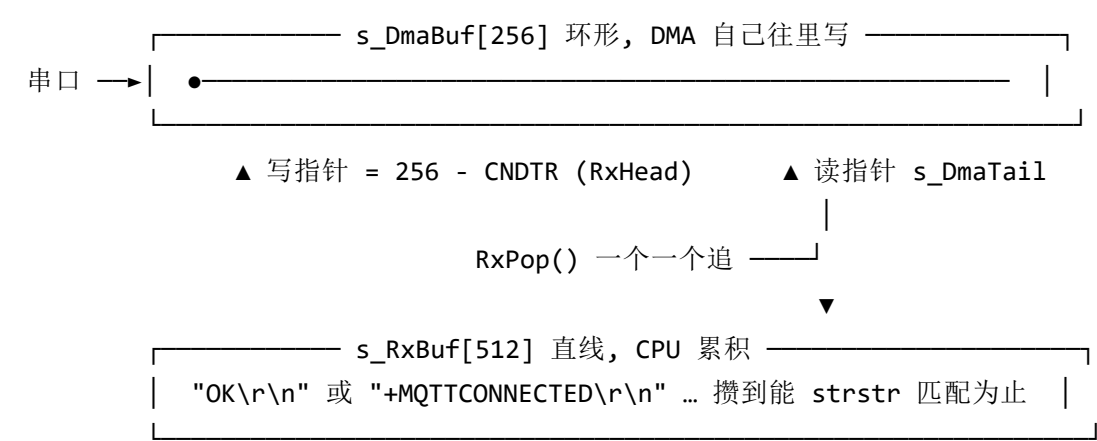
	普通模式 NORMAL	环形模式 CIRCULAR
CNDTR 归零后	通道 停止 （EN 自动清 0），传输结束	CNDTR 自动重装 成初值，继续跑
内存地址	走到缓冲区末尾就到头了	走到末尾 回卷 到开头，地址重来
什么时候用	长度已知、一次性 的一块（ADC 采 100 个点、发一帧定长数据）	长度未知、源源不断 的流（串口接收）
HAL 里怎么起	调一次 HAL_xxx_DMA() 搬一次，下次要重调	调一次就挂着，一直有效
传输完成中断	搬完那一次	每转完一圈来一次
本项目	没人用	接收路径 （ s_DmaBuf ）

本项目用到的 DMA 只有一路

项	配置	配在哪
通道	DMA1_Channel16 （F1 上 USART2_RX 的固定通道）	usart.c 的 HAL_UART_MspInit()
方向	DMA_PERIPH_TO_MEMORY	同上
模式	DMA_CIRCULAR	同上

项	配置	配在哪
数据宽度	外设、内存都是 BYTE	同上
优先级	DMA_PRIORITY_LOW （就它一个在用）	同上
缓冲	s_DmaBuf[256]	esp8266.c
谁启动	ESP8266_RxStart() ← ESP8266_Init()	esp8266.c
谁读进度	ESP8266_RxHead() 读 CNDTR	esp8266.c
中断	DMA1_Channel6_IRQn 优先级 5， 但传输完成中断被驱动关掉了	dma.c / usart.c

两级缓冲，各干各的



缓冲	大小	为什么这么大
s_DmaBuf	256 B	它只需要装下 两次搬运之间的突发 ：应用每 1 ms 搬一次，1 ms @115200 才 11 个字节，256 是 22 倍的余量
s_RxBuf	512 B	它要 累积到整条应答匹配完 ，最长的一条是 CWJAP 的回复（里面带 AP 信息），比一条命令的应答长得多

关键常量

宏	值	含义
ESP8266_DMA_BUF_SIZE	256	DMA 环形缓冲大小

宏	值	含义
ESP8266_RX_BUF_SIZE	512	解析缓冲大小（满 → ERR_BUF_FULL）

21.3 逐段详解

21.3.1 普通模式：一次搬运一个"窗口"

普通模式就像"搬箱子"：告诉 DMA"把这 256 个字节搬完"，搬完它就把通道关了。

```
/* 普通模式的语义(伪代码，本项目没用这条) */
HAL_UART_Receive_DMA(&huart, buf, 256); /* CNDTR = 256, 通道使能 */
/* ... 串口来一个字节 → DMA 搬一个 → CNDTR 减 1 ... */
/* CNDTR = 0 → 硬件置 TC 标志, 通道自动停 */

HAL_UART_Receive_DMA(&huart, buf, 256); /* ← 想再收, 必须重挂一次 */
```

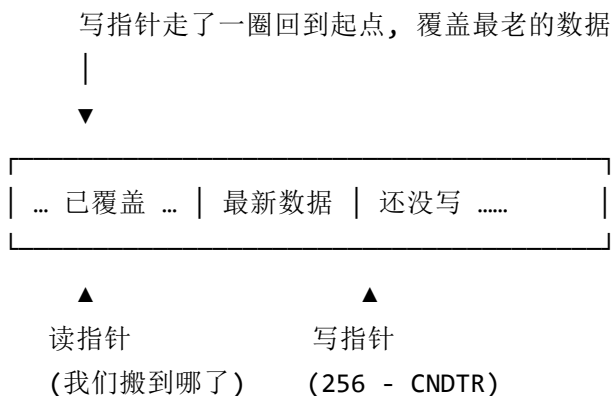
它的三个特点决定了它**不适合收"说不清什么时候来、来多少"的串口数据**：

1. **收满就停**。CNDTR 归零那一刻通道就没了，后面的字节没有人接 —— 串口只有一个字节的寄存器，下一个字节到来时寄存器还是满的，直接**溢出丢失**。
2. **要接着收就得重挂**。而"重挂"这个动作（HAL_UART_Receive_DMA 里要重新写 CNDTR、清标志、使能通道）需要 CPU 参与，最快也要几微秒 —— 而这期间来的字节，照样丢。**它丢的正好是两次窗口之间的那段**，也就是最容易出错、最不该丢的"边界字节"。
3. **你得知道什么时候收满**。于是必须开传输完成中断（TC）来通知，多一个中断、多一个 ISR 要写。（本项目反而要**关掉**这个中断，见 21.3.8。）

那普通模式什么时候是对的？**长度已知的一块数据**：ADC 采 100 个点、I2C 读一页 EEPROM、DMA 发送一帧定长报文 —— 搬完就结束，语义天然就是"一次性"。本项目的发送路径就是这类活，只是它连 DMA 都没用，直接中断发送（理由见 21.4 决策一）。

21.3.2 环形模式：窗口首尾相接

环形模式把缓冲区当成一个**圈**：CNDTR 减到 0 时自动重装成初值，内存地址回卷到开头，通道继续跑 —— **永远不停，永远有效**。



它换来的是**挂一次管到底**：RxStart() 调一次 HAL_UART_Receive_DMA()，之后不管程序在干什么、要收多少字节、什么时候来，环形缓冲都能接住——只要 CPU 在"写指针追上读指针"之前把数据搬走。

代价是**覆盖是静默的**：写指针绕一圈追上读指针时，老数据被直接盖掉，硬件不会报任何错（从 DMA 的角度看它一切正常）。丢没丢，只能靠自己算水位（见 21.5）。

21.3.3 CNDTR 是关键：它是"还剩几个才转一圈"

这是环形模式里最容易绕晕的一点。

普通模式里，CNDTR 的含义很直观："还剩几个没搬"。归零 = 搬完了。

环形模式里，CNDTR **永远不会归零后停住**：它减到 0 的那一刻立刻重装成初值（256），所以它读出来永远是 1 ~ 256。于是它的含义变成了：

CNDTR = 从当前位置写到"这一圈的终点"，还需要几个字节。

反过来就是本项目最核心的一行代码：

```
/* DMA 当前的写指针：环形模式下 "缓冲区大小 - CNDTR" 恒等于写位置 */
static uint16_t ESP8266_RxHead(void)
{
    return (uint16_t)(ESP8266_DMA_BUF_SIZE -
        __HAL_DMA_GET_COUNTER(ESP8266_UART.hdmarx));
}
```

- 一个字节都没收到 → CNDTR = 256 → 写指针 = 0。
- 收到 10 个字节 → CNDTR = 246 → 写指针 = 10。
- 转完一圈又收了 10 个字节 → CNDTR = 246 → 写指针 = 10（它在圈上，就是 10，不需要记录"第几圈"）。

`__HAL_DMA_GET_COUNTER()` 展开就是读一下 `CNDTR` 寄存器，不关中断、不改任何状态，可以随时读。

为什么这个公式在环形模式下成立：因为内存地址永远等于“写指针”，而写指针从 0 开始、每搬一个 +1、到末尾回卷到 0 —— 与 `CNDTR` 的递减是严格同步的。所以任何时候
已写字节数（模 256） = 256 - `CNDTR`。

21.3.4 配置： `usart.c` 里那几行

```
/* USART2_RX Init */
hdma_usart2_rx.Instance = DMA1_Channel6; /* F1 上 USART2_RX 只能挂这条 */
hdma_usart2_rx.Init.Direction = DMA_PERIPH_TO_MEMORY; /* 外设 → 内存 */
hdma_usart2_rx.Init.PeriphInc = DMA_PINC_DISABLE; /* 外设地址不增：永远是 DR */
hdma_usart2_rx.Init.MemInc = DMA_MINC_ENABLE; /* 内存地址递增，顺着往缓冲区里放 */
hdma_usart2_rx.Init.PeriphDataAlignment = DMA_PDATAALIGN_BYTE; /* 一个字节一个字节 */
hdma_usart2_rx.Init.MemDataAlignment = DMA_MDATAALIGN_BYTE;
hdma_usart2_rx.Init.Mode = DMA_CIRCULAR; /* ★ 环形：到末尾回卷，通道不停 */
hdma_usart2_rx.Init.Priority = DMA_PRIORITY_LOW; /* 就它一个在用，无所谓 */
if (HAL_DMA_Init(&hdma_usart2_rx) != HAL_OK)
{
    Error_Handler();
}

__HAL_LINKDMA(uartHandle, hdmarx, hdma_usart2_rx); /* 把 DMA 句柄挂到串口上 */
```

逐条说：

- `PeriphInc = DISABLE / MemInc = ENABLE` 是这一对的全部意义：**外设那头地址固定**（串口的数据寄存器就是那一个），**内存那头地址往前走**（不然 256 个字节会全盖在 `s_DmaBuf[0]` 上）。这两个配反了是初学者最常见的错：症状是“缓冲区里只有最后一个字节”。
- **对齐都是 BYTE**：串口一次请求就是一个字节，配成 `HALFWORD` 会一次搬两个字节、把两个字节拼成一个，收到的数据全是坏的。
- `DMA_CIRCULAR` **就是环形模式的开关**：这一个字段决定了 `CNDTR` 归零之后是“停”还是“重装继续”。
- `__HAL_LINKDMA`：HAL 库靠这个把 DMA 句柄和串口句柄绑在一起。少了它，`HAL_UART_Receive_DMA()` 找不到用哪条通道（`huart->hdmarx == NULL` 会直接返回错误）。
- **为什么优先级 5**（`dma.c` 里）：FreeRTOS 要求“会调 RTOS API 的中断”优先级数值 $\geq$ `configLIBRARY_MAX_SYSCALL_INTERRUPT_PRIORITY`（本工程 5）。本项目其实一个 `FromISR` 都没调，取 5 是**留余量**——将来真要在这个中断里唤醒任务，不用再回来改。

21.3.5 RxHead()：读 CNDTR 算出写指针

已经把公式讲完了（21.3.3），这里只说工程上要注意的两点：

- **CNDTR 是硬件在减的**，我们读它只是一个**快照**。读完之后到用它之前，DMA 可能又搬了几个字节——所以写指针是“至少写到这里了”。这个方向是**安全**的：你只会**少搬几个**（下次再搬），不会误以为有新数据。
- 反过来，**计算过程中的 CNDTR 读两遍不会一致**，所以只读一次、用它算出 head，不要“先判断再重读”。

21.3.6 RxPop()：读指针只在圈里追

```
/* 从环形缓冲取最多 max 字节到 dst，返回实际取了多少(0 = 没新数据).
 * 唯一推进读指针的地方 */
static uint16_t ESP8266_RxPop(uint8_t *dst, uint16_t max)
{
    uint16_t head = ESP8266_RxHead();
    uint16_t moved = 0;

    while (s_DmaTail != head && moved < max)
    {
        dst[moved++] = s_DmaBuf[s_DmaTail++];

        if (s_DmaTail >= ESP8266_DMA_BUF_SIZE)
        {
            s_DmaTail = 0;          /* ★ 读指针也回卷 — 这才叫“环” */
        }
    }

    return moved;
}
```

三个点值得记住：

1. **s_DmaTail 是全局的（在驱动里是文件级 static）**，因为它是“我读到哪了”的记忆，必须跨调用保留。head 是每次现算的。
2. **回卷要自己写**：s_DmaTail >= 256 就归零。这一句和 DMA 硬件那边的自动回卷是**两套独立的回卷**，一个在硬件里（写指针），一个在软件里（读指针），各回各的。
3. **循环条件 s_DmaTail != head**：这是“环形缓冲没有空/满之分”的直接体现——代码只判断“读指针是不是追上了写指针”（= 空了），**不判断“是不是追上了一圈”**（= 满了/被覆盖了）。因为一旦被覆盖，数据已经没了，代码也无力回天，不如不判（见 21.5）。

21.3.7 RxDrain() 和 ClearRxBuf()：两级缓冲之间怎么同步

```
/* 把新字节搬进解析缓冲，放不下就少搬点 — 读指针不能卡住，卡住就追不上 DMA
 * 了；满没满由 Exec() 判(ERR_BUF_FULL) */
static void ESP8266_RxDrain(void)
{
    uint16_t room = (uint16_t)(ESP8266_RX_BUF_SIZE - 1U - s_RxLen);

    s_RxLen = (uint16_t)(s_RxLen + ESP8266_RxPop(&s_RxBuf[s_RxLen], room));
}
```

- **RxPop(&s_RxBuf[s_RxLen], room)**：直接把新字节追加到解析缓冲已有的内容后面，room 是"还剩多少空位"。**上限约束在解析缓冲这边**（512），不是 DMA 那边。
- **放不下就少搬，绝不阻塞**：如果这里写成"缓冲满了就等上层来处理"，读指针 s_DmaTail 就会卡住不动——而 DMA 的写指针不会等它，转一圈就把老数据盖了。**卡住的读指针等于放弃整个环形缓冲**。所以宁可丢尾部几个字节，也要让读指针一直动。丢的后果由上层承担：Exec() 发现 s_RxLen 撑满就返回 ERR_BUF_FULL。

```
/* 清接收缓冲：DMA 里没搬走的和已搬进解析缓冲的都要清 — 只清后者，下次
 * Drain 又会被前者灌回来。
 * memset 不只是清内容：新追加的字节靠它保证末尾恒为 '\0'，strstr 才安全 */
static void ESP8266_ClearRxBuf(void)
{
    s_DmaTail = ESP8266_RxHead();          /* ★ 假装"DMA 里的我全读过了" */
    memset(s_RxBuf, 0, sizeof(s_RxBuf));
    s_RxLen = 0;
}
```

- **s_DmaTail = ESP8266_RxHead()** **这一句是环形缓冲特有的操作**：它不是"清空 DMA 缓冲"，而是**把读指针一把跳到写指针的位置**，等于宣布"圈里已有的这些我都不不要了"。普通模式里没有这个概念（那里就没有读指针）。
- 只清 s_RxBuf 是不够的：圈里没搬走的那部分，下一次 RxDrain() 又会灌回来——上一次命令的尾巴会被当成这一次的应答。
- 副作用：**模块空闲时主动上报的消息（+MQTTDISCONNECTED 之类）会被一起丢掉**，因为每条命令发出前都会清一次。

21.3.8 RxStart()：环形模式"重挂"必须先从停开始

```
static ESP8266_Status ESP8266_RxStart(void)
{
    /* 先停再开，这一步不能省：Receive_DMA 要求 RxState == READY，而重连时 DMA
     * 还在环形模式里跑(RxState=BUSY_RX)，不停掉就永远失败 —— 症状是"第一次
     * 失败之后永远失败"，看着像模块坏了。*/
    (void)HAL_UART_DMASStop(&ESP8266_UART);

    s_DmaTail = 0;

    if (HAL_UART_Receive_DMA(&ESP8266_UART, s_DmaBuf,
                             ESP8266_DMA_BUF_SIZE) != HAL_OK)
    {
        return ESP8266_ERR_AT;
    }

    /* 关掉 DMA 传输完成中断：环形模式下 HAL 的回调什么都不做，我们靠轮询
     * CNDTR 知道进度。*/
    __HAL_DMA_DISABLE_IT(ESP8266_UART.hdmarx, DMA_IT_TC);

    /* 串口自己的错误中断(HAL 在 Receive_DMA 里顺手开的 CR3.EIE)也要关：
     * 开了 USART2 中断之后，帧错/噪声/溢出任一标志都会让 HAL_UART_IRQHandler
     * 当成致命错误、直接中止 DMA 接收，接收就此全哑。模块每次复位吐的启动日志
     * 是 74880 波特，收进来就是一串帧错，正好踩中。我们靠轮询，不需要它。*/
    __HAL_UART_DISABLE_IT(&ESP8266_UART, UART_IT_ERR);

    return ESP8266_OK;
}
```

这段里每一句都和"环形模式"有关：

- **为什么要 DMASStop**：HAL_UART_Receive_DMA() 开头会检查 RxState，只有 READY 才肯配置。而环形模式的特点是**通道一直在跑**，RxState 一直是 BUSY_RX —— 不停掉它，重挂 100% 失败。**这个坑是环形模式专属的**：普通模式搬完自己就停了，RxState 会回到 READY，不会有这个问题。
- **s_DmaTail = 0**：这是重新开始，读指针也要归零，不然它会去追一个刚从头开始的写指针，读到错位的数据。
- **关 DMA 的 TC 中断**：环形模式下每转一圈都会来一次传输完成中断，而 HAL 的回调在这里什么都不做（它以为"搬完了"，其实只是转了一圈）。**我们的进度靠轮询 CNDTR，这个中断纯属白开。**
- **关串口的错误中断（UART_IT_ERR）★ 全驱动最不能删的一句**：HAL 在 HAL_UART_Receive_DMA() 里会顺手把串口的 CR3.EIE 打开。一旦 USART2_IRQHandler 生效，帧错 / 噪声 / 溢出任何一个标志

都会被 HAL 当成**致命错误**，直接中止 DMA 接收 —— 接收就此彻底哑掉。而 ESP8266 每次复位都会吐一段 **74880 波特的启动日志**，拿 115200 去收它就是一串帧错，**必定踩中**。我们靠轮询，根本不需要这个中断。

为什么"环形 + 轮询"要配套：环形模式解决了"硬件那头的窗口衔接"，轮询解决了"CPU 这头什么时候来取"。两者配在一起，接收路径上**一个中断都不需要** —— 连 DMA 通道自己的 TC 中断也是关掉的（DMA1_Channel16_IRQHandler 里的 HAL_DMA_IRQHandler() 因此在正常运行时永远不会被触发；NVIC 那边使能它是 CubeMX 生成的默认样子，留着不影响）。

21.3.9 如果换成普通模式，会发生什么

把 DMA_CIRCULAR 改成 DMA_NORMAL，其余代码不动，接收会变成这样：

收满 256 字节 → 通道停 → 后面的字节全丢

└─ 要重挂：HAL_UART_DMAStop + HAL_UART_Receive_DMA
(几微秒到几十微秒，这期间来的字节还是丢)

具体到本项目会出三类问题，每一类都很难查：

1. **AT 应答被切成两半：** +MQTTCONNECTED 这类长回复正好跨在窗口边界上时，前半段在上一个窗口、后半段丢在重挂的空窗里 → strstr 永远匹配不上 → 判超时 → 任务层判定"链路死了"去重连。**看着像模块不稳定。**
2. **RxHead() 的公式不再成立：**普通模式里 CNDTR 的含义是"这次搬运还剩几个"，搬完就是 0， $256 - 0 = 256$ 会被当成"写指针在 256"（越界），而它其实只是停了。**同一个函数在两种模式下的语义完全不同**，这也是为什么模式换了就不能只改一行。
3. **"什么时候该去重挂"没有干净的答案：**只能靠 TC 中断通知，于是又回到"必须有中断 + ISR 里不能干重活"的老路，还得处理"重挂期间来的字节"。**复杂度全回来了，而可靠性没有提升。**

一句话：环形模式不是"普通模式的加强版"，它们是给两种不同的数据形态准备的 —— **流用环形，块用普通。**

21.4 关键设计决策

决策一：接收用环形 DMA，发送用中断 —— 不对称是故意的

	接收	发送
数据形态	长度未知、源源不断的流	长度已知的一条命令（≤256 字节）
该用什么	环形 DMA（挂一次管到底，CPU 忙也不丢）	中断发送就够（115200 下 22 ms 发完）
本项目	DMA1_Channel16 + DMA_CIRCULAR	HAL_UART_Transmit_IT + 轮询 gState

给发送也配一条 DMA 通道的收益是零（省不下多少 CPU，还多一套中断），而接收**非 DMA 不可**（关中断 4 ms 读 DHT11 时不能让字节丢）。**判断依据是数据形态，不是"哪个更高级"**。

决策二：靠轮询 CNDTR，不靠中断和回调

- 收到多少字节：读 CNDTR（21.3.3）。
- 什么时候搬：Exec() 等应答的循环里每 1 ms 搬一次。
- 因此**不需要** TC 中断（还得主动关掉）、**不需要** HAL_UART_RxCpltCallback（那是全局弱符号，多串口会打架）、**不需要**串口的错误中断（还得主动关掉，见 21.3.8 ★）。

代价是**响应有最多 1 ms 的延迟**（要等下一次搬运）。对 AT 应答来说完全无所谓 —— 反正 Exec() 本来就每毫秒搬一次。

决策三：不用 IDLE 中断，用"定期搬"

另一条常见路线是：环形 DMA + **串口空闲中断（IDLE）**，一帧数据收完（总线空闲）时进中断，在 ISR 里通知任务。它的好处是"不用轮询，帧边界天然明确"。

本项目没走这条路，理由有三条：

1. **IDLE 的语义是"总线空闲了一个字节时间"**，对 AT 这种"应答中间可能有停顿"的文本协议，一帧会被切成好几段 IDLE，反而要在 ISR 里做"是不是真收完了"的判断。
2. **ISR 里不能干重活**（解析、strstr、打印），只能置个标志让任务去干 —— 而任务本来就在 Exec() 里 1 ms 转一圈等着，**轮询的写法更短、更直白**。
3. 还要多配一个中断使能、多一处"忘了清 IDLE 标志就再也不进中断"的坑。

什么时候该改成 IDLE：如果哪天接收路径不再是"发一条等一条"，而是要被动接收**不定长、不定时的**数据流（比如真正跑 TCP 透传时解析 +IPD），那时 IDLE 中断才划算。

决策四：DMA 只许写 s_DmaBuf ，解析缓冲永远由 CPU 搬

DMA 的目的地永远是那个 256 字节的环形缓冲， s_RxBuf （512 字节的解析缓冲）只由 CPU 通过 RxPop() 写。这条界线带来两个好处：

- 1. DMA 写的地方永远是"我们只读"的，不用担心"上一轮解析到一半，DMA 又把新字节盖进来"。
- 2. 解析缓冲可以按"最长的应答"来定大小（512），而不必按"可能积压多少数据"来定 —— 两个缓冲各自的最优大小本来就差一个数量级（11 字节/ms 的突发 vs 最长的一条回复）。

决策五：缓冲区大小 = "CPU 最长能停顿多久"的换算

s_DmaBuf 定 256 字节不是随手写的，它等于给 CPU 留了：

$$256 \text{ 字节} \div 11.52 \text{ 字节/ms} \approx 22 \text{ ms}$$

也就是说，只要 CPU 每 22 ms 至少来搬一次，环形缓冲就追得上。本项目的搬运间隔是 1 ms（ Exec() 的循环），余量 22 倍。如果哪天改成"每 100 ms 搬一次"，这条等式就是你该先算的东西。

21.5 已知局限与注意事项

环形缓冲不保证"一定不丢"

这是最容易被误解的一点。环形缓冲保证的是"硬件那头的窗口衔接不会丢"（普通模式重挂的空窗），但它不保证 CPU 来得及搬。写指针绕一圈追上读指针时，老数据被静默覆盖：DMA 不会置任何错误标志，因为从它的角度看一切正常。

什么时候真的会丢（都在本项目里成立）：

场景	会不会丢	为什么
CPU 关中断 4 ms 读 DHT11	不丢	DMA 不需要 CPU，4 ms 才 46 个字节，圈子装得下
任务卡在别的地方 20 ms	不丢（但快了）	20 ms $\approx$ 230 字节，接近 256 的余量
任务卡 100 ms	丢	写指针早绕回来了，只是代码不知道
解析缓冲满了 （ ERR_BUF_FULL ）	丢	RxDrain() 少搬 $\rightarrow$ 读指针不动 $\rightarrow$ 下一圈覆盖

场景	会不会丢	为什么
一条应答正好占了整整 256 字节	可能被误判成"没有新数据"	转整圈后 <code>head == tail</code> , 看起来像空

最后一条是这个设计的固有盲点（下面单独说），前几条则是"CPU 停顿超过 22 ms"的不同版本 —— **丢不丢，取决于你停顿多久，而不是取决于代码写得对不对。**

head == tail 的二义性

环形缓冲用两个指针表达状态，**"空"和"恰好满一圈"是同一个状态**（`s_DmaTail == head`）。本项目把这一状态一律当成"空"。

后果：如果两次搬运之间 DMA 恰好写了**整数倍的 256 字节**，代码会以为"没有新数据"，那一段就被跳过了。实际上要凑齐这个条件并不容易（要正好写满一圈、而且恰好在这一刻被搬运撞上），而且 256 字节的整圈对应 22 ms 的连续数据 —— 真发生了，说明系统已经不正常了。

正规的修法是让 DMA 去写 `256 + 1` 字节的缓冲（多留一个字节做哨兵），或者维护一个"圈数"计数。本项目没做，因为**代价大于收益**：它要动 `s_DmaBuf` 的大小、`RxHead()` 的公式和 `RxPop()` 的判断，换来的是"运行已经不正常时不容易察觉的丢包" —— 不划算。

其他不能忘的

1. `RxStart()` 里必须先 `HAL_UART_DMAStop()` —— 环形模式下通道一直在跑，`RxState == BUSY_RX`，不先停就重挂不上（症状是"第一次失败之后永远失败"）。
2. `__HAL_UART_DISABLE_IT(&ESP8266_UART, UART_IT_ERR)` **不能删** —— 删了接收早晚全哑（74880 波特的启动日志会被当成帧错，HAL 直接中止 DMA 接收）。
3. **DMA 的 TC 中断在环形模式下没有意义**，主动关掉；`DMA1_Channel16_IRQHandler` 平时不会触发。
4. **读指针绝不能被"挡住"**：`RxPop()` 搬不动就少搬，`RxDrain()` 放不下就不放 —— 任何形式的"读指针卡住等条件"都会把整个环形缓冲变成垃圾场。
5. `s_DmaBuf` **千万别让 DMA 之外的人写**：它是 DMA 的目的地，CPU 只读不写。往里写会破坏"写指针 = `256 - CNDTR`"这个前提。

21.6 动手改这里

改缓冲区大小

`esp8266.h` 里的 `ESP8266_DMA_BUF_SIZE` 和 `ESP8266_RX_BUF_SIZE`，改完直接编。两点注意：

- 别小于 CPU 的最大停顿：按 $\text{大小} \div 11.52 \text{ 字节/ms}$ 算，256 字节 = 22 ms。
- s_RxBuf 要装得下最长的一条应答。改小了，CWJAP 的回复会报 ERR_BUF_FULL。

换成普通模式（只在特殊场景才该做）

1. usart.c：hdma_usart2_rx.Init.Mode = DMA_NORMAL；
2. 接收逻辑**必须重写**：RxHead() 的公式不再成立（普通模式 CNDTR = 0 表示停了，不是"写指针在 256"），要改成"用 TC 中断/标志判断这次搬完了，然后重挂"。
3. 重挂要在**下一个字节到来之前完成**（87 μ s），这件事本身就很难保证——这也是为什么串口收数据几乎没人用普通模式。

什么场景该用普通模式：你需要的是"精确收满 N 个字节就停"，比如和外设做定长帧的通信。**串口收 AT 应答不属于这一类。**

加 IDLE 中断（想被动收不定长数据时）

1. usart.c 里打开 IDLE 中断：__HAL_UART_ENABLE_IT(&huart2, UART_IT_IDLE)；
2. 在 USART2_IRQHandler 里**先读 SR 再读 DR** 清掉 IDLE 标志（这是 F1 的固定套路，少读一次就再也不进中断了）。
3. ISR 里只置标志 + 通知任务，解析仍然在任务里做（别在 ISR 里 strstr、打印）。
4. 记得**不能和现在那句关 UART_IT_ERR 冲突**：IDLE 是另一条使能位，互不影响，但改完要重新确认错误中断仍然是关的。

换串口 / 换 DMA 通道

- 换串口要改 esp8266.h 的 ESP8266_UART，然后按 F1 的**硬连线表**找到对应的 DMA 通道（USART1_RX = DMA1_Channel5, USART3_RX = DMA1_Channel3 ...），在 usart.c 里改 Instance 并同步 NVIC。
- **F1 不能像 F4 那样随便挑通道**，这是硬件的限制。

用调试器看它在动没动

悬着程序看三个数就能确认环形接收在正常工作：

看什么	正常的表现
hdma_usart2_rx.Instance->CNDTR	模块说话时在 变小 ，然后跳回 256（转圈）
s_DmaTail	每次 RxDrain() 之后 追上去
DMA1_Channel16->CCR 的 CIRC 位	必须是 1（环形）。是 0 说明模式配错了

21.7 一句话总结

环形模式和普通模式的区别只有一句话：CNDTR 减到 0 之后，通道是"停下来等 CPU 来重挂"还是"自己重装、回卷、继续跑"。串口收数据是长度未知的流，只能选后者 —— 挂一次管到底，CPU 忙那 4 ms 也不丢字节；代价是"写指针追上读指针时静默覆盖"，所以要靠轮询 CNDTR 把数据及时搬走。

三件最容易被误解的事：

- 1. 环形模式下 CNDTR 不是"还剩几个"，而是"从当前位置到这一圈终点还有几个"。所以
写指针 = 缓冲区大小 - CNDTR —— 这行公式就是整个接收层的地基。
- 2. 环形模式必须"先停再重挂"：通道一直在跑，RxState 恒为 BUSY_RX，不 DMAStop 就永远挂不上。普通模式没有这个问题。
- 3. 环形缓冲不等于"不丢"：它只解决了硬件窗口的衔接，CPU 停顿超过"缓冲区 ÷ 字节率"（本项目 256 ÷ 11.52 ≈ 22 ms）照样覆盖，而且没有任何错误标志。

附：相关文件一览

文件	看什么
Core/Src/usart.c	hdma_usart2_rx 的七个字段（方向 / 自增 / 对齐 / CIRCULAR ）、__HAL_LINKDMA
Core/Src/dma.c	MX_DMA_Init()：DMA1 时钟、DMA1_Channel16_IRQn 优先级 5
Core/Src/stm32f1xx_it.c	DMA1_Channel16_IRQHandler()（正常运行时不会被触发）
BSP/ESP8266/esp8266.c	RxStart() / RxHead() / RxPop() / RxDrain() / ClearRxBuf()
BSP/ESP8266/esp8266.h	ESP8266_DMA_BUF_SIZE（256）、ESP8266_RX_BUF_SIZE（512）
BSP/dht11/dht11.c	__disable_irq() 后面那 4 ms —— 环形缓冲存在的理由